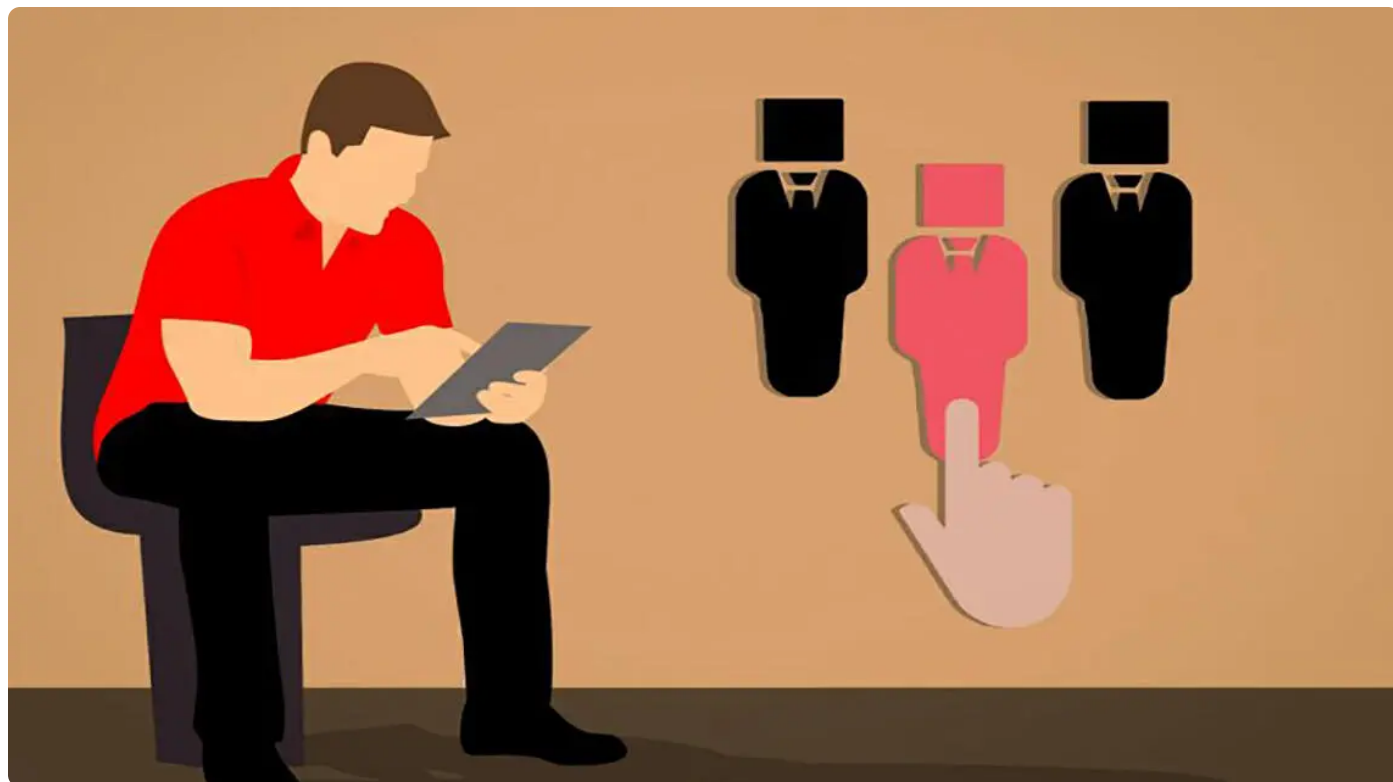


## 05 | 限流：别说算法了，就问你“阈值”怎么算？

2023-6-26 邓明



你好，我是大明。今天我们来聊一聊微服务架构下的限流功能。

熔断、降级和限流是最常见的三种微服务架构可用性保障措施。和熔断、降级比起来，限流要更加复杂一些。大部分情况下，面试官面试限流就是随便问问算法，最多就是问问 BBR 之类的动态算法。但是有一个问题，很多人都答不好，就是**限流需要确定一个流量阈值，这个阈值该怎么算？**

今天我就带你深入讨论限流的这个问题。

### 前置知识

限流是通过限制住流量大小来保护系统，它尤其能够**解决异常突发流量打崩系统的问题**。例如常见的某个攻击者攻击你维护的系统，那么限流就能极大程度上保护住你的系统。

要想全面掌握限流这个知识点，我们需要深入理解限流的算法、对象，以及限流后的做法。下面我们一个一个来看。

### 算法

限流算法也可以像负载均衡算法那样，划分成静态算法和动态算法两类。

静态算法包含令牌桶、漏桶、固定窗口和滑动窗口。这些算法就是要求研发人员提前设置好阈值。在算法运行期间它是不会管服务器的真实负载的。

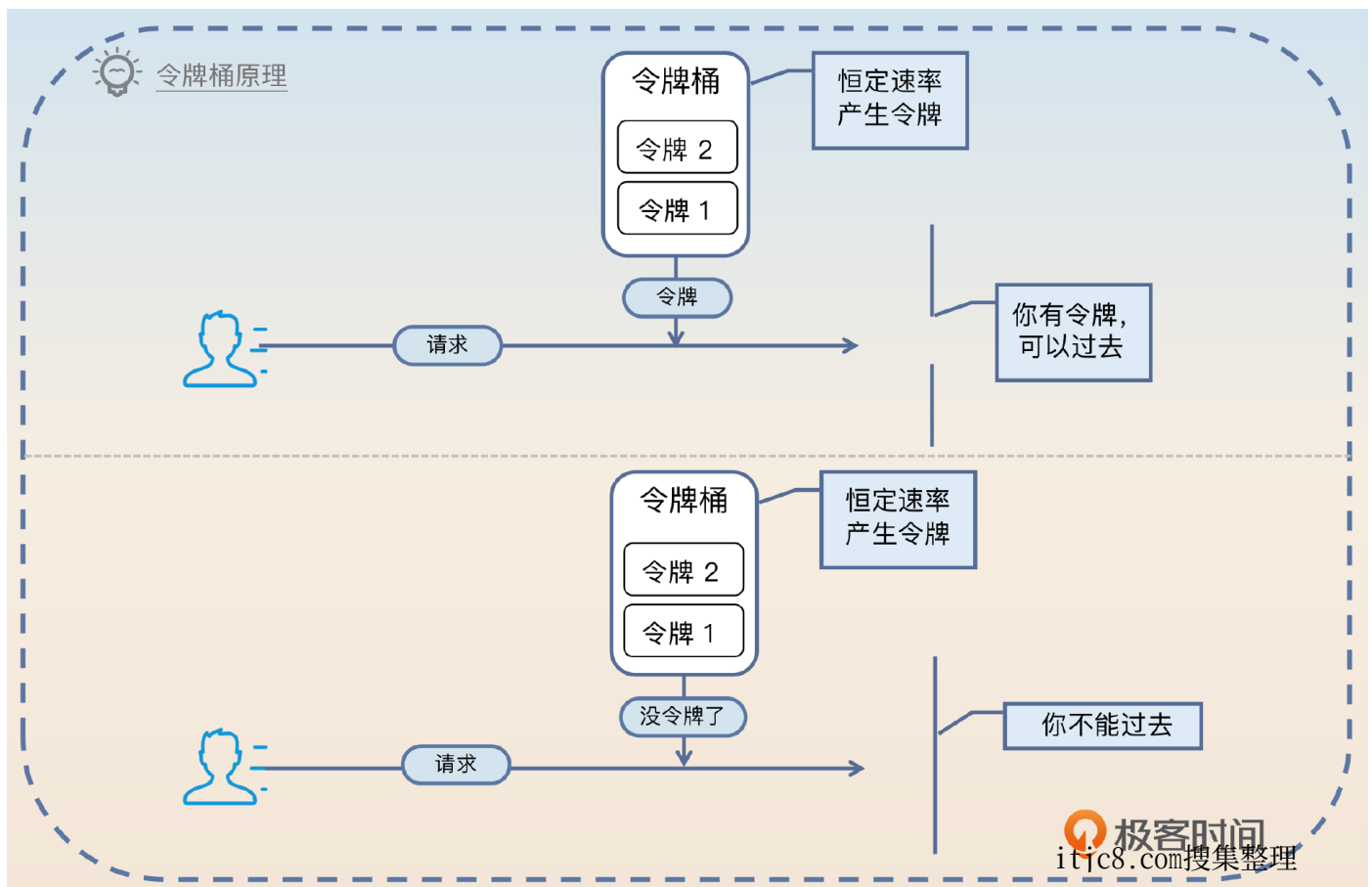
动态算法也叫做自适应限流算法，典型的是 BBR 算法。这一类算法利用一系列指标来判定是否应该减少流量或者放大流量。动态算法和 TCP 的拥塞控制是非常接近的，只不过 TCP 控制的是报文流量，而微服务控制的是请求流量。

除了我这里列举的算法，你也可以考虑参考熔断和降级里面的思路，选用一些指标来设计自己的限流算法。例如你的业务需要很多内存，那么你可以根据剩余空闲内存来判断要不要执行限流。

下面我们就从静态算法中的令牌桶看起，掌握限流中常见的算法。

## 令牌桶

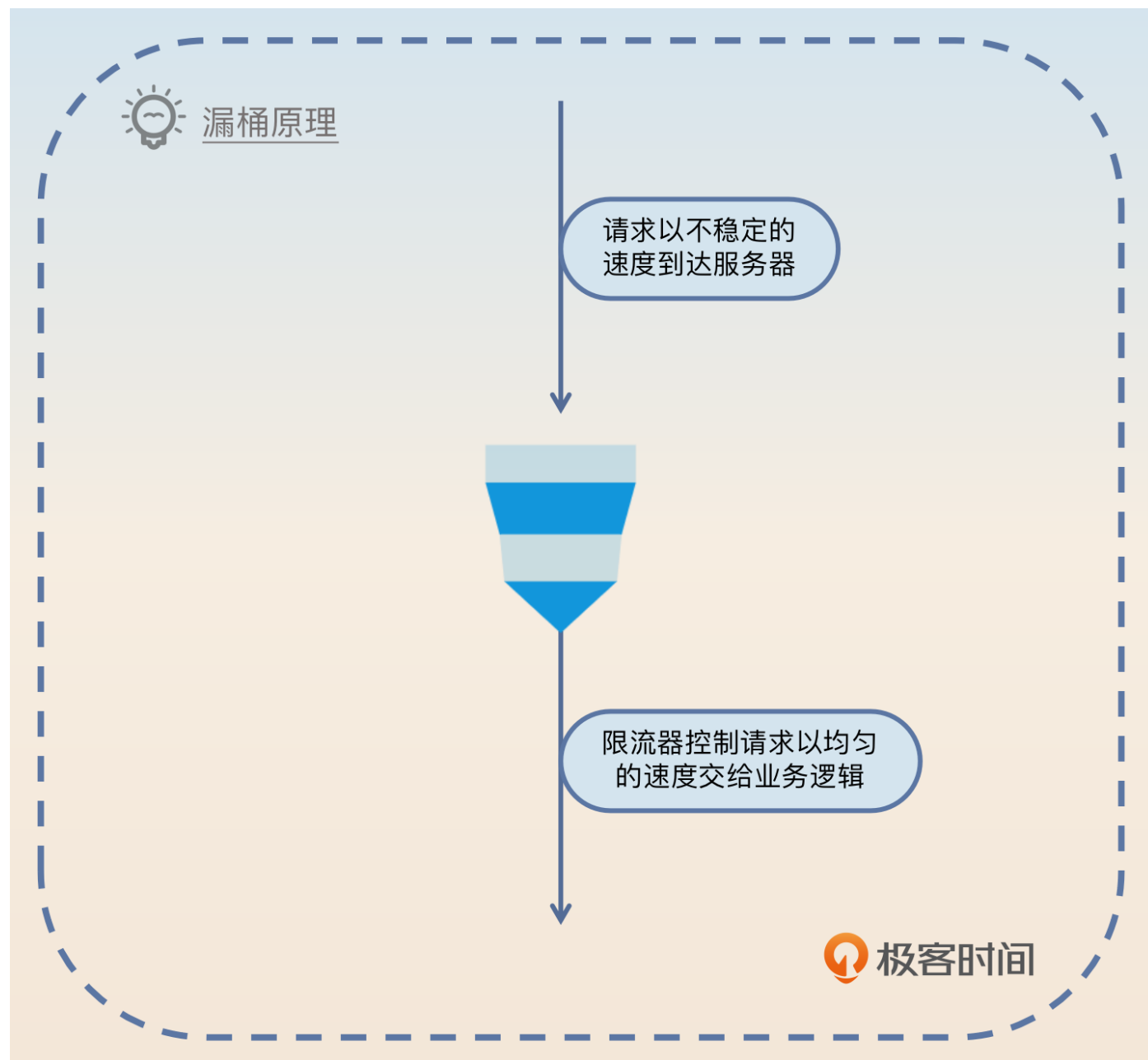
系统会以一个恒定的速率产生令牌，这些令牌会放到一个桶里面，每个请求只有拿到了令牌才会被执行。每当一个请求过来的时候，就需要尝试从桶里面拿一个令牌。如果拿到了令牌，那么请求就会被处理；如果没有拿到，那么这个请求就被限流了。



你需要注意，**本身令牌桶是可以积攒一定数量的令牌的**。比如说桶的容量是 100，也就是这里面最多积攒 100 个令牌。那么当某一时刻突然来了 100 个请求，它们都能拿到令牌。

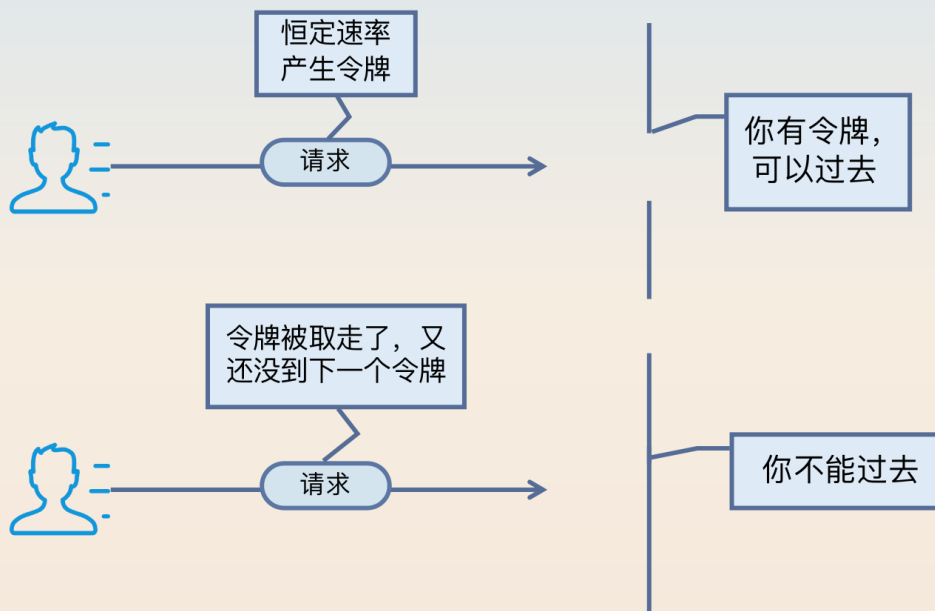
## 漏桶

漏桶是指当请求以不均匀的速度到达服务器之后，限流器会以固定的速率转交给业务逻辑。



某种程度上，你可以将漏桶算法看作是令牌桶算法的一种特殊形态。你将令牌桶中桶的容量设想为 0，就是漏桶了。

💡 无容量的令牌桶约等于漏桶

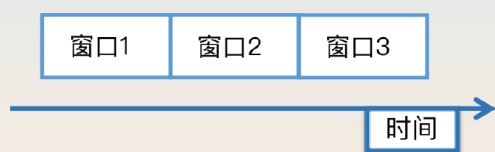


极客时间

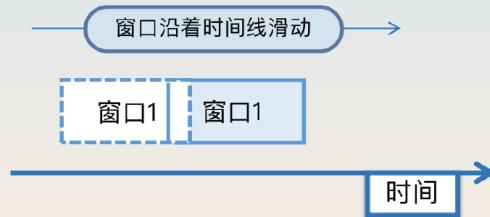
所以你可以看到，在漏桶里面，令牌产生之后你就需要取走，没取走的话也不会积攒下来。因此漏桶是绝对均匀的，而令牌桶不是绝对均匀的。

## 固定窗口与滑动窗口

💡 固定窗口



💡 滑动窗口



极客时间

固定窗口是指在一个固定时间段，只允许执行固定数量的请求。比如说在一秒钟之内只能执行 100 个请求。

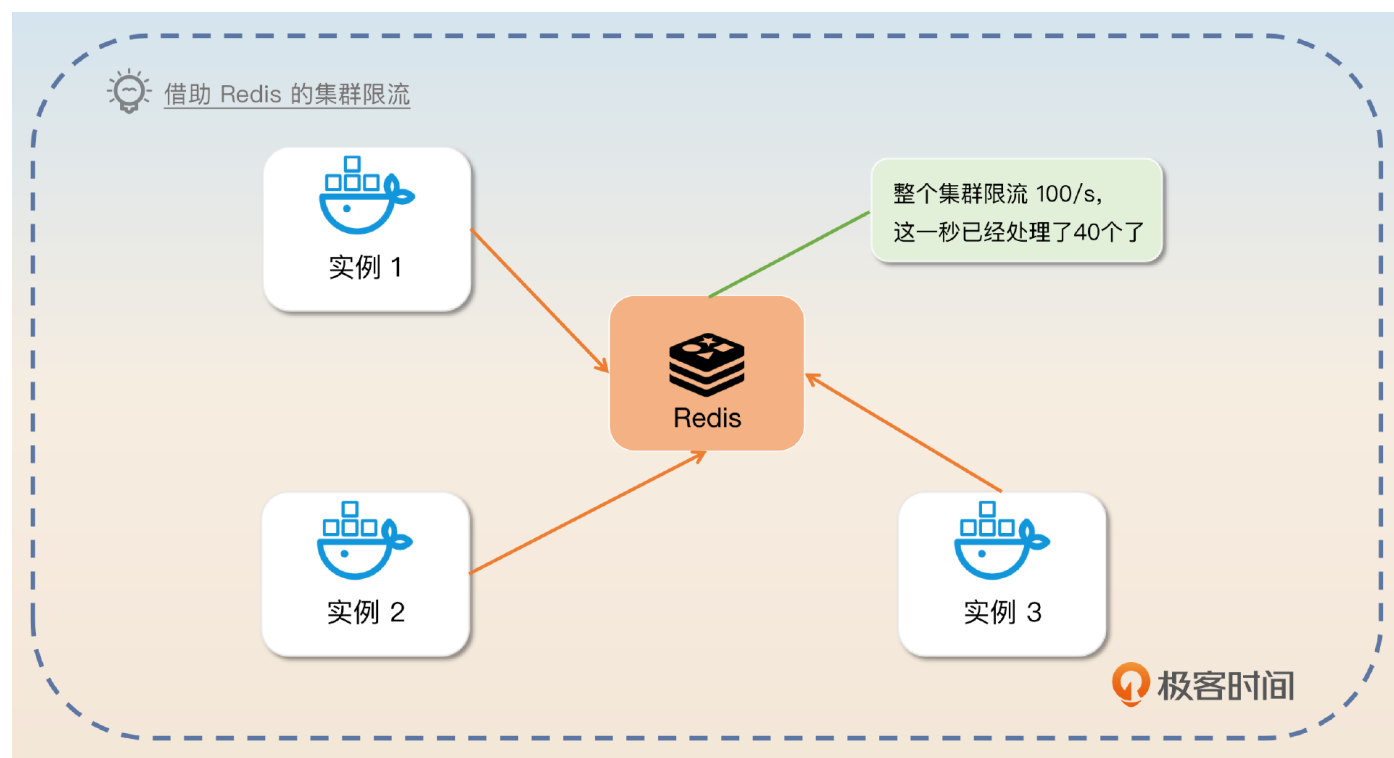
滑动窗口类似于固定窗口，**也是指在一个固定时间段内，只允许执行固定数量的请求**。区别就在于，滑动窗口是平滑地挪动窗口，而不像固定窗口那样突然地挪动窗口。

假设窗口大小是一分钟。此时时间是  $t_1$ ，那么窗口的起始位置是  $t_1 - 1$  分钟。过了2秒之后，窗口大小依旧是1分钟，但是窗口的起始位置也向后挪动了2秒，变成了  $t_1 - 1$  分钟 + 2 秒。这也就是滑动的含义。

## 限流对象

此外我们还要进一步考虑限流对象，也就是针对什么来进行限流。

从单机或者集群的角度看，可以分为单机限流或者集群限流。集群限流一般需要借助 Redis 之类的中间件来记录流量和阈值。换句话说，就是你需要用 Redis 等工具来实现前面提到的限流算法。当然如果是利用网关来实现集群限流，那么可以摆脱 Redis。

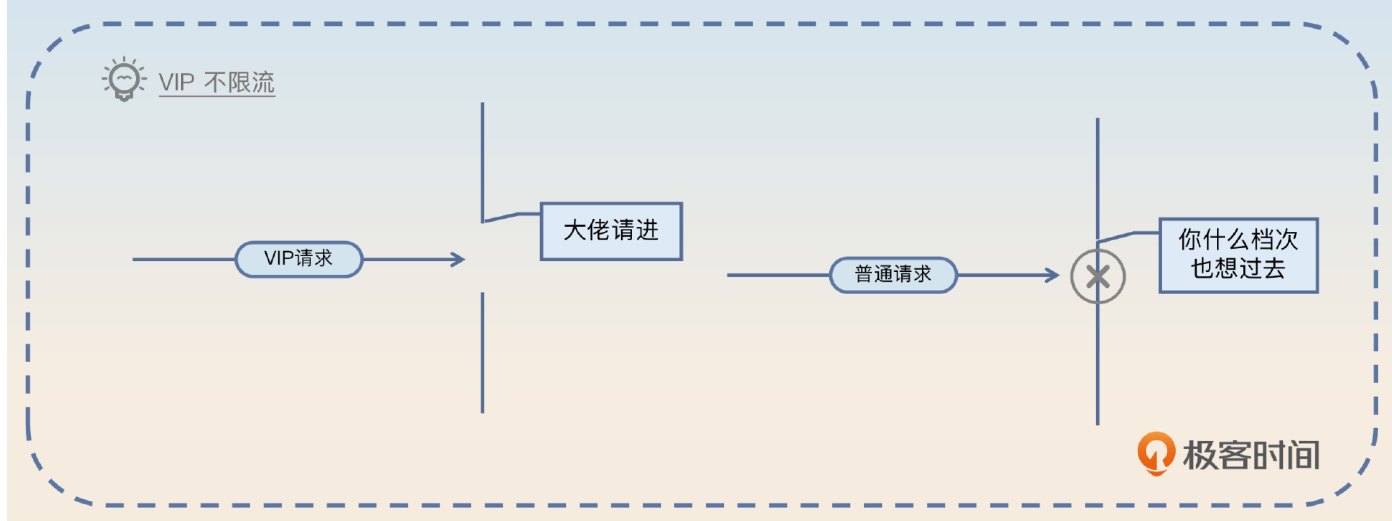


针对业务对象限流，这一类限流对象就非常多样。

VIP 用户不限流而普通用户限流。

针对 IP 限流。用户登录或者参与秒杀都可以使用这种限流，比方说设置一秒钟最多只能有 50 个请求，即便考虑到公共 IP 的问题，正常的用户手速也是没那么快的。

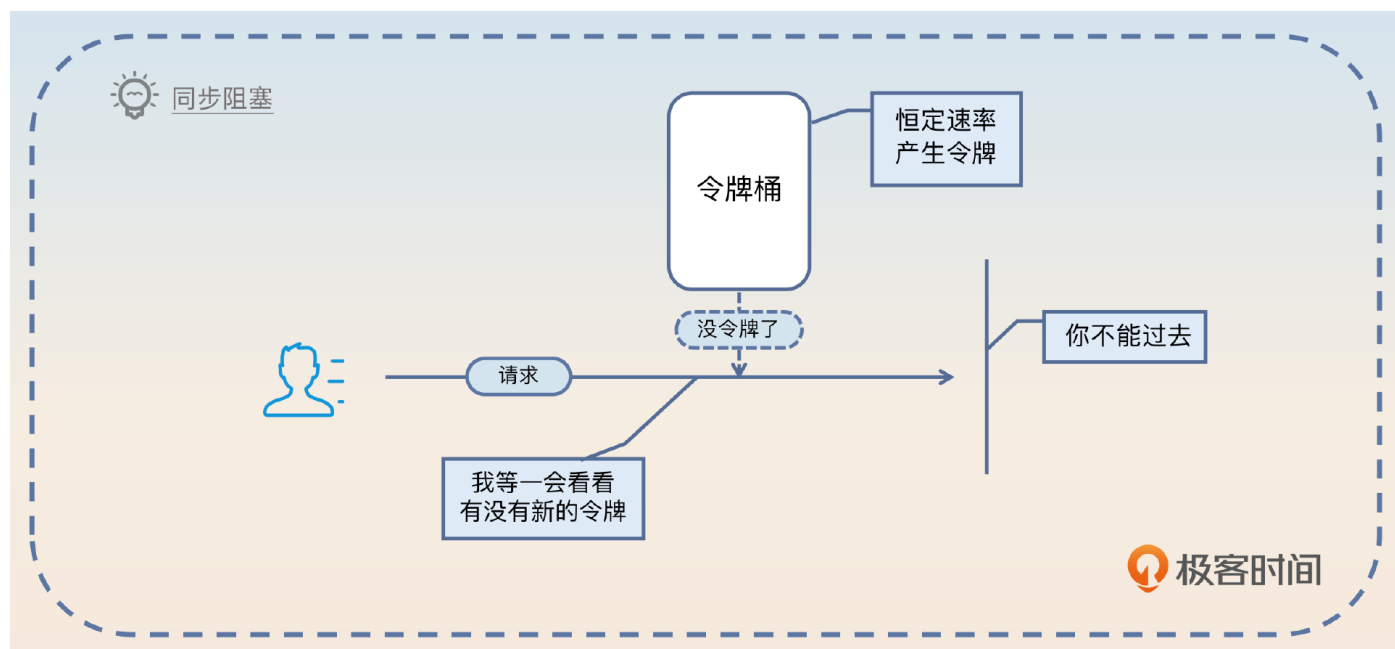
针对业务 ID 限流，例如针对用户 ID 进行限流。



## 限流后的做法

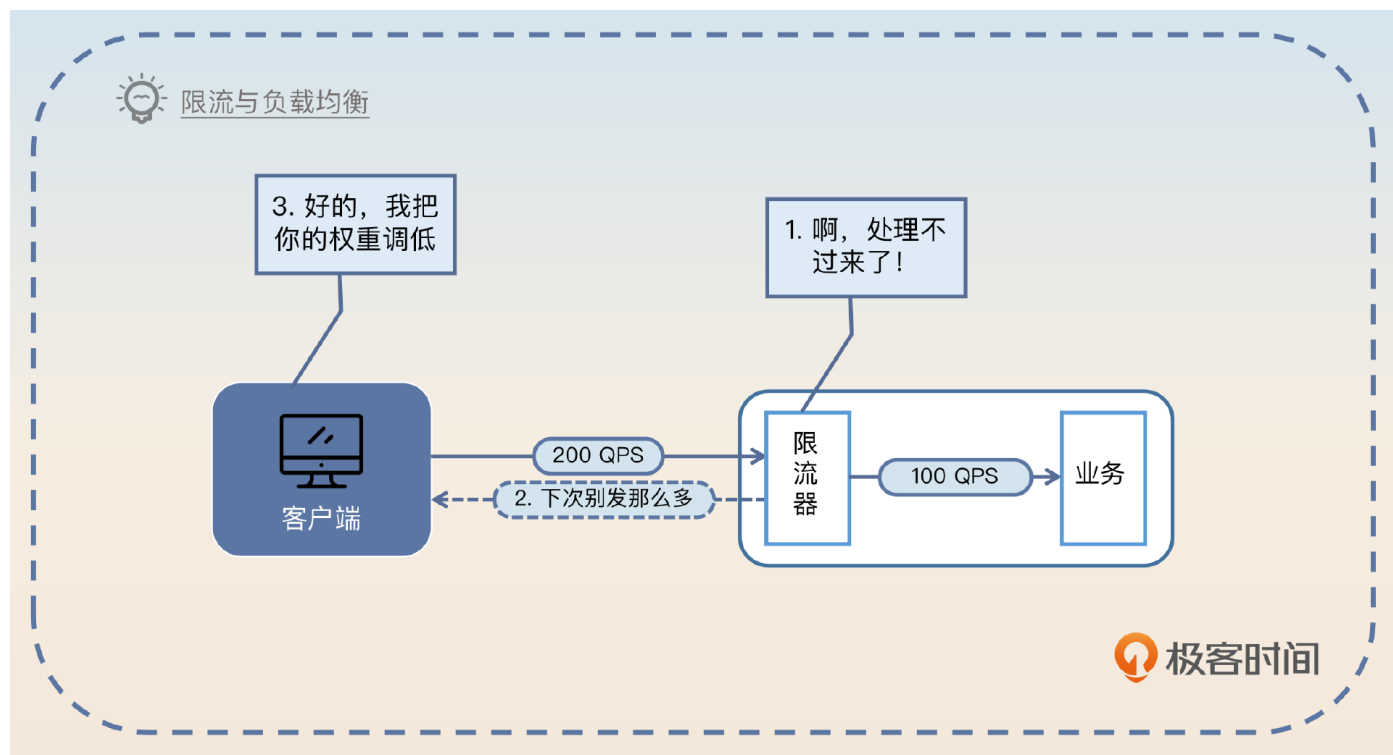
即使一个请求被限流了，那么我们也可以设计一些精巧的方案来处理。

同步阻塞等待一段时间。如果是偶发性地触发了限流，那么稍微阻塞等待一会儿，后面就有极大的概率能得到处理。比如说限流设置为一秒钟 100 个请求，恰好来了 101 个请求。多出来的一个请求只需要等一秒钟，下一秒钟就会被处理。但是要注意控制住超时，也就是说你不能让人无限期地等待下去。



同步转异步。这里我们又一次看到了这个手段，它是指如果一个请求没被限流，那就直接同步处理；而如果被限流了，那么这个请求就会被存储起来，等到业务低峰期的时候再处理。这个其实跟降级差不多。

调整负载均衡算法。如果某个请求被限流了，那么就相当于告诉负载均衡器，应该尽可能少给这个节点发送请求。我在熔断里面给你讲过类似的方案。不过在熔断里面是负载均衡器后续不再发请求，而在限流这里还是会发送请求，只是会降低转发请求到该节点的概率。调整节点的权重就能达成这种效果。



## 面试准备

理论上，你要能够说出各种算法的基本原理。但动态算法，比如 BBR，就不作硬性的要求了。这主要是因为 BBR 的原理和实现都很有难度，大多数微服务框架都没提供 BBR 的限流器实现。不过还是那句老话，你要是有时间和精力，还是可以了解一下 BBR 的基本原理。

有些时候面试官可能会让你手写限流算法，那么漏桶、令牌桶、滑动窗口和固定窗口这几个算法你都要能写出来，至少要能把基本思路写清楚。如果你还有时间和精力，那么我建议你为一些开源框架提供限流插件，比如说为 gRPC 提供各种限流算法实现的插件。

你可能会说，现在开源的那么多，你写出来的插件还有人用吗？大概率没人用，但是你的目标也不是让人用，而是**作为一个证据**，来证明你懂限流，你很熟悉 gRPC，你喜欢开源。



开源不是为了 star，是为了证据

1. 我精通限流、gRPC，热爱开源

2. 我不信

3. 不信看我 GitHub

极客时间

除了这些基本的知识，在面试前，你还需要了解清楚你们公司使用限流的情况。正常来说，核心 HTTP 请求和核心服务都应该使用限流来保障系统的可用性。对于每一个限流，你都要了解这些信息。

限流的阈值是多少，为什么设定成这个阈值？

被限流的请求会被怎么处理，是直接拒绝还是阻塞直到超时，还是转为异步处理？

同样，面试限流的最好策略就是为自己打造一个掌握了高可用微服务架构的人设。而限流就是你在提高系统可用性时的一个具体策略。如果你前面和面试官已经聊到了熔断和降级，那么就可以直接把话题引导到限流上。比如：

在讨论对外的API，如HTTP 接口或者公共API时，可以强调使用限流来保护系统。

在讨论TCP拥塞控制时，你可以提起在服务治理上限流也借鉴了TCP拥塞控制的一些思想。

在讨论Redis或者类似产品的时候，你可以提你用 Redis 实现过集群限流。

如果你维护的服务或者接口还没有使用限流来保护系统，那么你就可以考虑加上限流。而为了确定具体的阈值，你可以尝试对接口进行压力测试，找准限流的阈值。这个也是需要你通过实践来加深印象、把握细节的。

## 基本思路

如果面试官问到了限流，那么你就可以先阐述限流的总体目标，然后回答前置知识里面的三个点：算法、限流对象和限流后的做法，最后再把话题引到计算阈值上。

限流是为了保证系统可用性，防止系统因为流量过大而崩溃的一种服务治理手段。从算法上来说，有令牌桶、漏桶、固定窗口和滑动窗口算法。还有动态限流算法，或者说自适应限流算法，比较有名的就是参考了 TCP 拥塞控制算法 BBR 衍生出来的算法，比如说 B 站开源的 Kratos 框架就有一个实现。这些算法之间比较重要的一个区别是**能否处理小规模突发流量**。

从限流对象上来说，可以是集群限流或者单机限流，也可以是针对具体业务来做限流。比如说在登录的时候，我们经常针对 IP 进行限流。又或者在一些增值服务里面，非付费用户也会被限流。

触发限流之后，具体的措施也可以非常灵活。被限流的请求可以同步阻塞一段时间，也可以考虑同步转异步。如果负载均衡算法灵活的话，也可以做一些调整，减少发到该节点的概率。

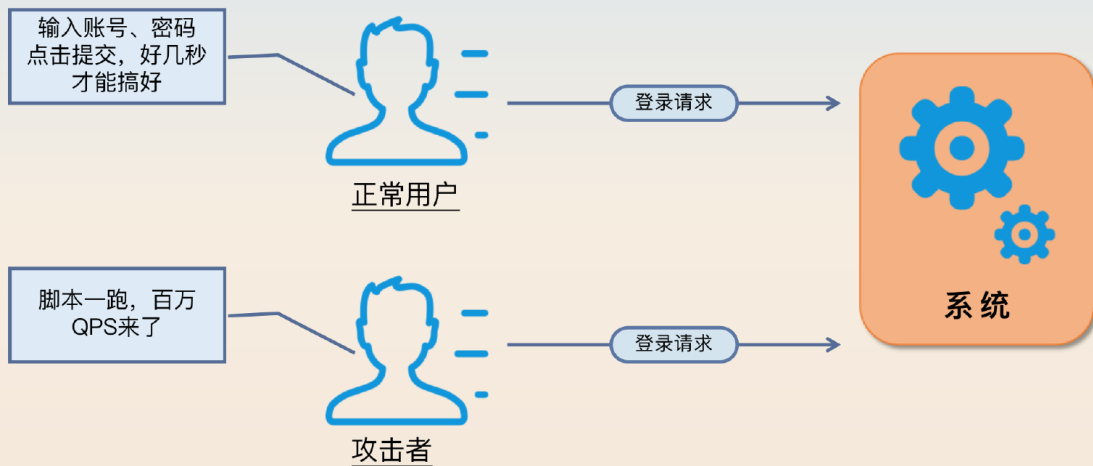
用好限流的一个重要前提是能够设置准确的阈值，例如每秒钟限制在 100 个请求还是限制在 200 个请求。如果阈值过低，那么系统资源就容易闲置浪费；如果阈值太高，那么系统可能撑不住那么多流量，导致崩溃。

同时你还要补充一个简单的例子，关键词是**IP限流**。你也可以考虑使用你的真实案例。

我在我们公司的登录接口里面就引入了限流机制。正常情况下，一个用户在一秒钟内最多点击一次登录，所以针对每一个 IP，我限制它最多只能在一秒内提交 50 次登录请求。这个 50 充分考虑到了公共 IP 的问题，正常用户是不可能触发这个阈值的。这个限流虽然很简单，但是能够有效防范一些攻击。不过限流再怎么防范，还是会出现系统撑不住流量的情况。



正常用户是比较难弄出来高 QPS 的



极客时间

注意在上面的回答里，我们没有说任何的细节，只是宽泛地介绍了一下限流，那么面试官接下来大概会问每一个算法、不同的限流对象，以及限流后的不同做法的细节。这部分你按照前置知识里面的内容来回答就可以。

同时，如果你开源了一个限流的仓库，你可以一起介绍一下。

我在 GitHub 上有一个开源仓库，里面放了我为 gRPC 实现的各种限流算法，包括基于 Redis 实现的集群限流版本。

如果你是跨语言面试，比如你是 Python 转 Go，你就可以强调一下，这个原本在我司是 Python 写的，后来我用 Go 又写了一遍。如果你有进行一些改进，你可以把你具体改进的内容表述出来。

接下来是一些对限流的深入讨论，这部分内容能让你刷出不少的亮点。

## 突发流量

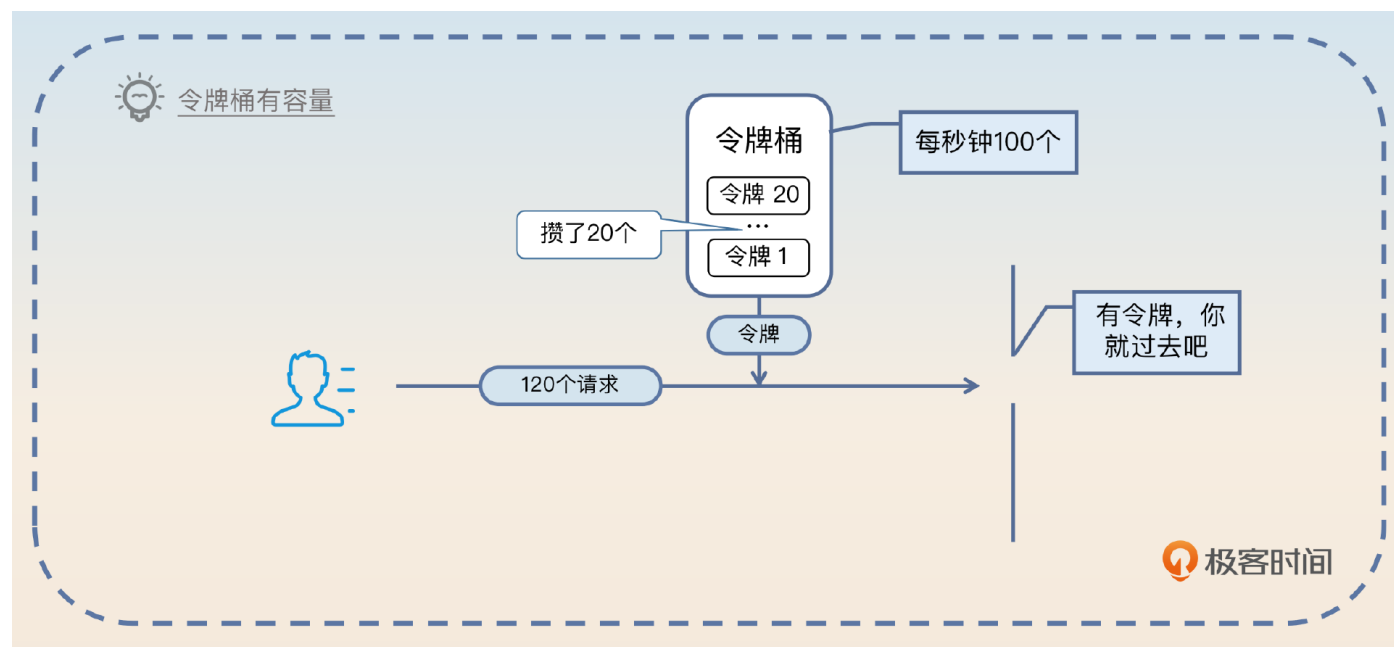
前面我们提到了“算法之间比较重要的一个区别是能否处理小规模突发流量”，就是为这个部分的详细阐述留下了一个引子。

假如说正常你的限流是一秒钟 100 个请求，但是如果某一秒钟来了 101 个请求，你依旧会觉得第 101 个请求应该尽可能处理掉。在这种场景下，漏桶是做不到的，因为漏桶是非常均匀

的。一秒钟 100 个请求在漏桶里面就是 10 毫秒一个请求，绝对不会多也不会少。

而令牌桶就能够处理。比如说令牌桶产生令牌的速率是 100 个每秒，但是桶的容量是 20 个，那么也就是说某一秒钟内，最多可以处理 120 个请求。

$20 \text{ (前一秒攒的令牌)} + 100 \text{ (当下这一秒)} = 120$

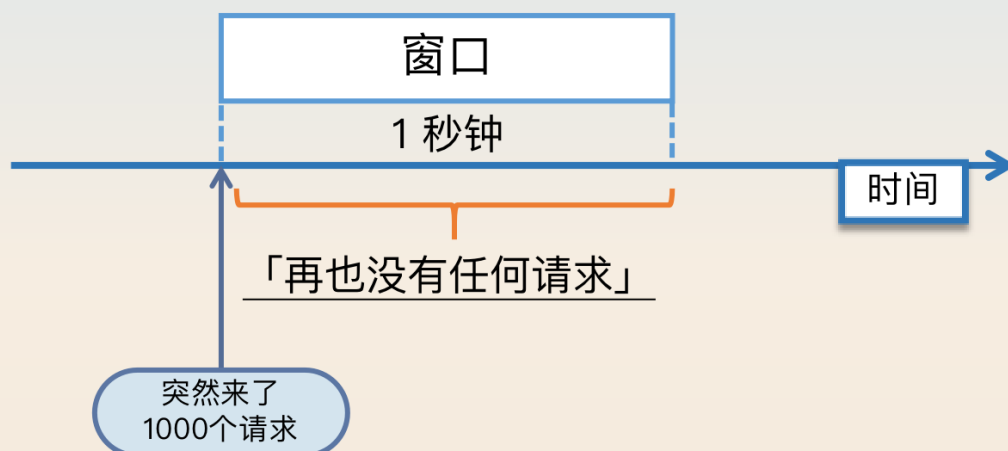


固定窗口和滑动窗口则有另外一个类似的问题，就是毛刺问题。

假如一个窗口大小是一分钟 1000 个请求，你预计这1000 个请求会均匀分散在这一分钟内。那么有没有可能第一秒钟就来了 1000 个请求？当然可能。那当下这一秒系统有没有可能崩溃？自然也是可能的。



请求集中在窗口的一边



所以固定窗口和滑动窗口的窗口时间不能太长。比如说以秒为单位是合适的，但是以分钟作为单位就是不合适的。

那么在面试官问到，或者你在介绍了漏桶或令牌桶算法之后，就可以补充这一段。

漏桶算法非常均匀，但是令牌桶相比之下就没那么均匀。令牌桶本身允许积攒一部分令牌，所以如果有偶发的突发流量，那么这一部分请求也能得到正常处理。但是要小心令牌桶的容量，不能设置太大。不然积攒的令牌太多的话就起不到限流效果了。例如容量设置为 1000，那么要是积攒了 1000 个令牌之后真的突然来了 1000 个请求，它们都能拿到令牌，那么系统可能撑不住这突如其来的 1000 个请求。

## 请求大小

刚刚我们讨论的限流是针对请求的个数进行的，但并没有考虑到另一个非常关键的问题，就是请求的大小。我在负载均衡里曾提过到这个问题，就是负载均衡算法基本上都没有考虑请求所需的资源。同理在限流算法也是如此。

限流是针对请求个数进行的，那么显然，如果有两台实例，一台实例处理的都是小请求，另一台实例处理的都是大请求，那么都限流在每秒 100 个请求。可能第一台实例什么问题都没有，而第二台实例就崩溃了。

所以如果面试官问到为什么使用了限流，系统还是有可能崩溃，或者你在负载均衡里面聊到了请求大小的问题，都可以这样来回答，关键词是**请求大小**。

限流和负载均衡有点儿像，基本没有考虑请求的资源消耗问题。所以负载均衡不管怎么样，都会有偶发性负载不均衡的问题，限流也是如此。例如即便我将一个实例限制在每秒 100 个请求，但是万一这个 100 个请求都是消耗资源很多的请求，那么最终这个实例也可能会承受不住负载而崩溃。动态限流算法一定程度上能够缓解这个问题，但是也无法根治，因为一个请求只有到它被执行的时候，我们才知道它是不是大请求。

以上就是我们在回答限流相关问题时的基本思路，如果可以回答出来，基本上就可以拿到一个70分的成绩，你满意吗？相信你和我一样，还想要更加出类拔萃一点，那这时候就要从计算阈值上面下功夫了。

## 计算阈值

在面试限流的基本回答里面，你已经主动提起了限流阈值难以确定。那么不出所料，面试官就会问你怎么确定阈值。又或者你使用了限流的不同案例，那么面试官也会问你限流的阈值是怎么确定的。

总体上思路有四个：看服务的观测数据、压测、借鉴、手动计算。

看服务的性能数据属于常规解法，基本上就是看业务高峰期的 QPS 来确定整个集群的阈值。如果要确定单机的阈值，那就再除以实例个数。所以你可以这样来回答，关键词是**业务性能数据**。

我们公司有完善的监控，所以我可以通过观测到的性能数据来确定阈值。比如说观察线上的数据，如果在业务高峰期整个集群的 QPS 都没超过 1000，那么就可以考虑将阈值设定在 1200，多出来的 200 就是余量。

不过这种方式有一个要求，就是服务必须先上线，有了线上的观测数据才能确

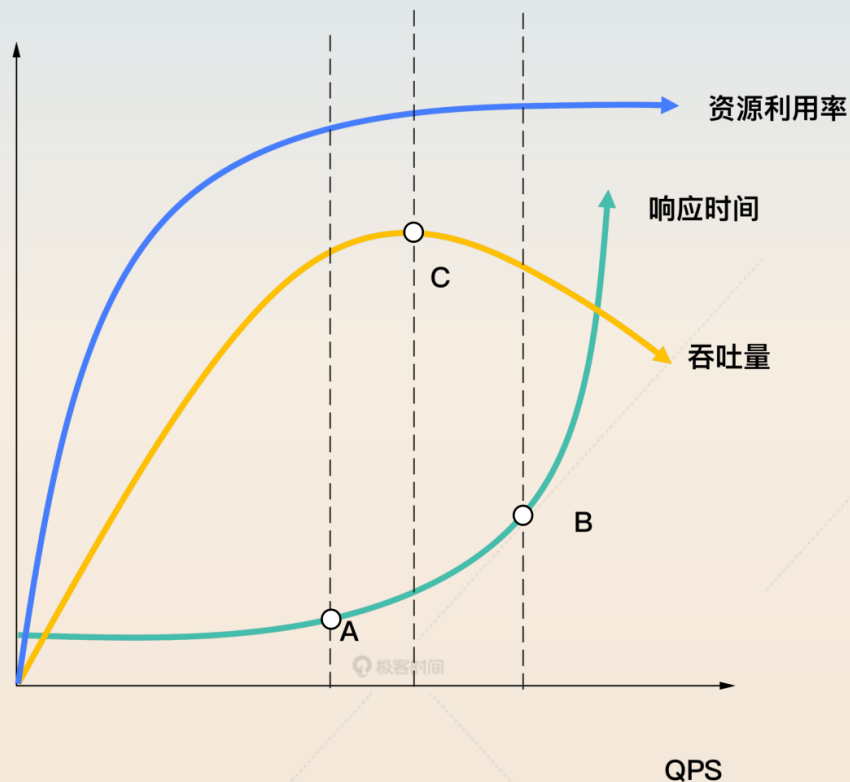
定阈值。并且，整个阈值很有可能是偏低的。因为业务巅峰并不意味着是集群性能的瓶颈。如果集群本身可以承受每秒3000个请求，但是因为业务量不够，每秒只有 1000 个请求，那么我这里预估出来的阈值是显著低于集群真实瓶颈 QPS 的。

注意你在回答的时候也解释了这种方法的缺陷，这算是一个小亮点。然后我们可以继续讨论其他的思路，关键词是**压测**。

不过我个人觉得，最好的方式应该是在线上执行全链路压测，测试出瓶颈。即便不能做全链路压测，也可以考虑模拟线上环境进行压测，再差也应该在测试环境做一个压力测试。

在这个回答里面你其实已经回答出了最正确的思路：做压测，而且你要强调全链路压测。理由很简单，限流针对的是线上环境，那么自然要尽可能模拟线上环境。最符合这个要求的就是全链路压测了，它就是直接在线上环境执行的，因此结果也是最准的。

然后你需要进一步解释，怎么利用压测结果。大部分性能测试的结果类似于图片里展示的这样，当然你是不太可能搞出来那么优雅的图形，多少会有些偏差。



从理论上来说，你可以选择 A、B、C 当中的任何一个点作为你的限流的阈值。

A是性能最好的点。A 之前 QPS 虽然在上升，但是响应时间稳定不变。在这个时候资源利用率也在提升，所以选择 A 你可以得到最好的性能和较高的资源利用率。

B是系统快要崩溃的临界点。很多人会选择这个点作为限流的阈值。这个点响应时间已经比较长了，但是系统还能撑住。选择这个点意味着能撑住更高的并发，但是性能不是最好的，吞吐量也不是最高的。

C是吞吐量最高的点。实际上，有些时候你压测出来的 B 和 C 可能对应到同一个 QPS 的值。选择这个点作为限流阈值，你可以得到最好的吞吐量。

你在回答怎么选之前，最好给面试官比划一下上面这张图中的三条曲线，然后解释这三个点，口诀就是**性能A、并发B、吞吐量C**。

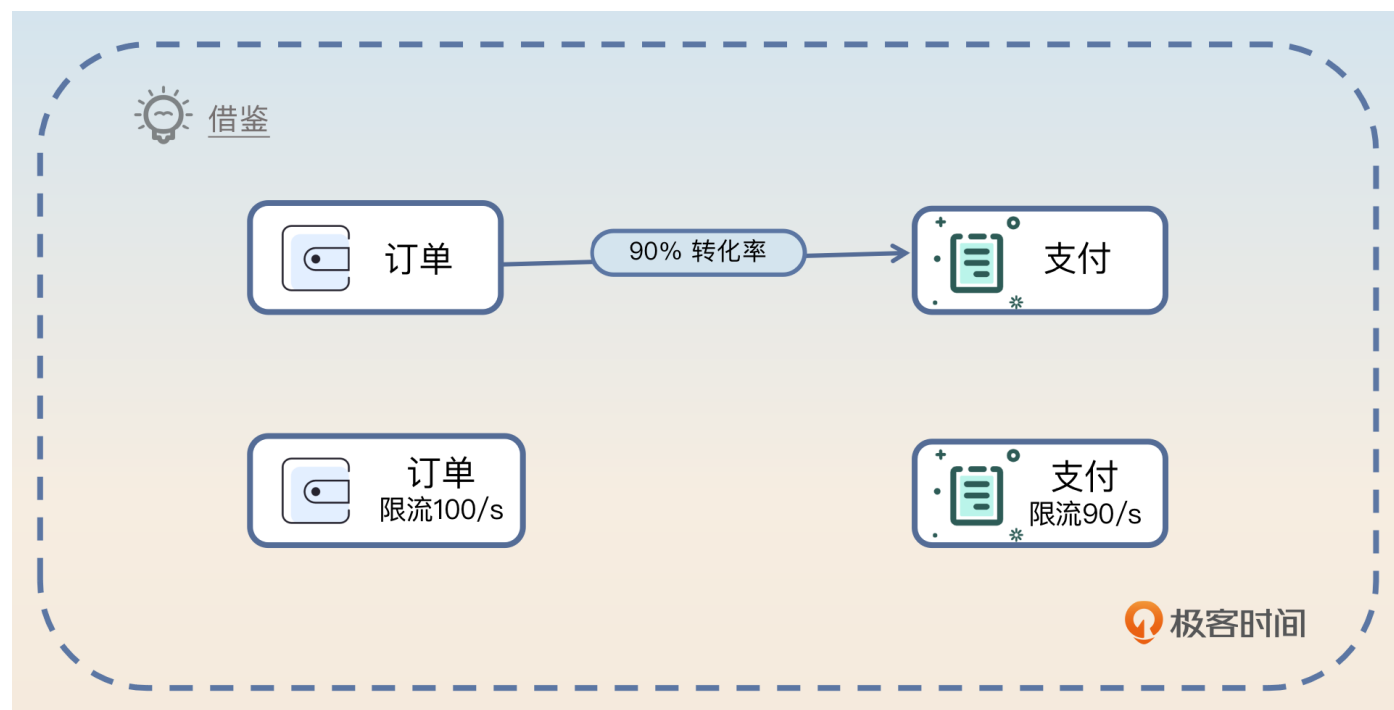
综合来说，如果是性能苛刻的服务，我会选择 A 点。如果是追求最高并发的服务，我会选择 B 点，如果是追求吞吐量的服务，我会选择 C 点。

面试官多半会杠你，压力测试特别难，或者有些服务根本测不了，那你怎么办。这个时候，你需要说点正确但没用的废话，关键词**压测是基操**。你在表述的时候语气要委婉，态度要坚决。

一般我会认为一家公司应该把压测作为提高系统性能和可用性的一个关键措施，毕竟没有压测数据，性能优化和可用性改进也不知道怎么下手。所以我还是比较建议尽可能把压测搞起来，反正压测这个东西是迟早要有的。

然后你就要转过话头，顺着面试官的话往下说，讨论真的做不了压测的时候怎么确定阈值。关键词就是**借鉴**。

不过如果真的做不了，或者来不及，或者没资源，那么还可以考虑参考类似服务的阈值。比如说如果 A、B 服务是紧密相关的，也就是通常调用了 A 服务就会调用 B 服务，那么可以用 A 已经确定的阈值作为 B 的阈值。又或者 A 服务到 B 服务之间有一个转化关系。比如说创建订单到支付，会有一个转化率，假如说是 90%，如果创建订单的接口阈值是 100，那么支付的接口就可以设置为 90。



这个时候面试官可能会继续问：如果我这是一个全新的业务呢？也就是说，你都没得借鉴。这个时候就只剩下最后一招了——**手动计算**。

实在没办法了，就只能手动计算了。也就是沿着整条调用链路统计出现了多少次数据库查询、多少次微服务调用、多少第三方中间件访问，如Redis，Kafka等。举一个最简单的例子，假如说一个非常简单的服务，整个链路只有一次数据库查询，这是一个会回表的数据库查询，根据公司的平均数据这一次查询会耗时 10ms，那么再增加 10 ms 作为 CPU 计算耗时。也就是说这一个接口预期的响应时间是 20ms。如果一个实例是 4 核，那么就可以简单用  $1000\text{ms} \div 20\text{ms} \times 4 = 200$  得到阈值。

这个时候你还可以进一步补充一些手动计算要考虑的事情。

手动计算准确度是很差的。比如说垃圾回收类型语言，还要刨除垃圾回收的开销，相当于 200 打个折扣。折扣多大又取决于你的垃圾回收频率和消耗。

最后再升华一下主题。

最好还是把阈值做成可以动态调整的。那么在最开始上线的时候就可以把阈值设置得比较小。后面通过观测发现系统还很健康，就可以继续上调阈值。

## 面试思路总结

这节课我们讨论了限流的主要问题，包括限流算法，限流对象以及限流之后的做法。在讨论怎么计算阈值的问题时，你尤其要**记住里面提到的 ABC 三个点**。不仅仅是面试中，在你实际工作中也用得上的。

我也再次强调一下，你应该在面试前尽可能在公司里面应用一下限流，同时尝试做一做压测。如果公司没有这种压测的环境，那么这正好是你刷 KPI 的机会。你把压测环境准备好，流程标准化之后，这件事情本身也可以作为你面试时候的一个亮点。

同样地，这里有我整理的思维导图，你可以参考。



## 思考题

最后你来思考几个问题。

针对 IP 限流是一个非常常见的限流方案，那么怎么获得用户的 IP 呢？尤其是在请求经过了网关的情况下，怎么避免自己拿到的是网关的 IP？

我在阈值里面提到的 ABC 三个点，你能说出你的业务应该使用哪个点吗？

欢迎你把你的答案分享在评论区，也欢迎你把这节课的内容分享给需要的朋友，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

## 精选留言 13